Investment Comparator

Process Book — COM-480 Data Visualization

Nicholas Brandstätter (330394)

Marko Djuric (330515)

Toufan Kashaev (347341)

EPFL, Spring 2026

Project Website: burek.vercel.app

Project Overview

The Investment Comparator is an interactive web-based tool designed to democratize financial analysis by making sophisticated investment comparison accessible to beginners. Built on 11 years of real historical data (2014–2024), the platform enables users to explore and contrast the performance of 21 assets across three major categories: cryptocurrencies (Bitcoin, Ethereum, Solana, Cardano, Polygon, Chainlink, Avalanche), traditional markets (S&P 500, NASDAQ, Russell 2000, FTSE 100, Nikkei 225, DAX, QQQ, VTI, US Real Estate), and commodities (Gold, Silver, WTI Oil, Copper, Natural Gas).

Our target audience—university students, beginner investors, and finance enthusiasts—often finds existing tools like Portfolio Visualizer or Bloomberg Terminal either prohibitively expensive or intimidatingly complex. We address this gap by providing an intuitive, visually engaging alternative that transforms raw financial data into actionable insights through interactive visualizations.

The tool goes beyond simple price comparison by incorporating educational elements (tooltips, guided tours, help sections), advanced risk metrics (15+ professional indicators), portfolio optimization (Markowitz efficient frontier), and temporal analysis (animated evolution timeline). Users can answer questions like “Is crypto worth the risk compared to gold, stocks, and real estate?” through hands-on exploration rather than passive consumption of financial advice.

Design Evolution: From Concept to Implementation

Initial Wireframes and Design Decisions

Our design process began with identifying the core user need: comparing investment performance across diverse asset classes in an intuitive way. Early wireframes focused on three main views: a main comparator with normalized performance chart, a risk analysis dashboard, and a DCA calculator.

Key design decisions made early in the process:

- **Dark Theme:** Chosen for reduced eye strain during extended analysis sessions and professional aesthetic matching financial terminals
- **Base-100 Normalization:** Essential for comparing assets with vastly different price scales (Bitcoin at $40,000 vs Gold at $2,000)
- **Category Organization:** Grouping by Crypto/Stocks/Commodities helps users navigate 21 assets without overwhelming choice
- **Color Coding:** Blue (Crypto), Green (Stocks), Yellow (Commodities) provides instant visual categorization across all pages

Evolution from Milestone 2 to Final Version

Professor feedback on Milestone 2 emphasized: “Making the experience even more guided for beginners, for example by adding tooltips directly next to the financial metrics.” This critical input shaped our development priorities:

> **Key Changes from Milestone 2:**
>
> - Added 17 educational tooltips explaining financial terms (Sharpe Ratio, Volatility, Max Drawdown, DCA, etc.)
> - Added 8-step guided tour with react-joyride for first-time users
> - Added help modal with FAQ (10 questions) and quick start guide
> - Upgraded “Preview” features (correlation matrix, risk metrics) from static placeholders to interactive D3.js visualizations
> - Expanded DCA calculator from 6 to 21 functional assets with real price data

- Completed Portfolio Optimizer with efficient frontier (was “Coming Soon”)
- Added PDF export functionality for sharing analysis

Iterative Refinements

Feature	Initial State	Final State	Reason
Historical Events	Disabled button	Interactive markers + impact zones, crosshairs, click-to-lock	Contextualizes performance data
Correlation Matrix	Static image	Interactive D3.js heatmap with category filters	Enables relationship discovery
Risk Metrics	3 basic indicators	15+ professional metrics (expand/collapse)	Serves beginners and advanced users
Scatter Plot	Simple plot	Zoom/pan, Voronoi, convex hulls	Professional exploration tool

Interactive Visualizations: Technical Deep Dive

The project features six custom D3.js visualizations, each designed to reveal different aspects of investment data and demonstrate mastery of interactive data visualization principles.

1. Animated Asset Evolution Timeline (Showcase)

Hans Rosling-style animated bubble chart (2014–2024)

Visual Encoding: X = Volatility (risk), Y = Annual return, Size = Return magnitude, Color = Category

Key Interactions:

- Play/Pause/Reset with 1x/2x/4x speed control
- Year scrubber for direct navigation
- Click any bubble to reveal its historical trail path
- Staggered transitions (15ms per bubble) with 1400ms d3.easeCubicInOut
- Year counter animates with d3.interpolateNumber tween
- Trail drawing uses stroke-dashoffset animation

Design Challenge: Initial implementation re-rendered the entire SVG on each year change, causing jerky motion. We solved this by separating static setup (axes, grid) from dynamic update (bubbles, trails, labels), using D3's enter/update/exit pattern with proper data keys.

Course Concepts: Storytelling with data (Lecture 12), D3.js transitions (Lecture 4), Interaction techniques (Lecture 5)

2. Efficient Frontier (Portfolio Optimizer)

Markowitz Mean-Variance Optimization with Monte Carlo simulation

Visual Encoding: Blue curve = Efficient frontier, Green dot = Max Sharpe, Purple dot = Min variance, Yellow = Individual assets, Green dashed = Capital Market Line

Technical Implementation:

- 8000 random portfolio simulations with directed weight sampling
- Covariance matrix from 60 months of return data
- Frontier extraction via volatility bucketing (80 bins)

- Click any frontier point to see its weight allocation
- Horizontal bar chart shows portfolio composition

Design Challenge: Without a QP solver library, we used Monte Carlo + directed sampling (30% biased toward concentrated portfolios) to adequately explore the feasible set. Results match analytical solutions within 2%.

Course Concepts: Position encoding (Lecture 6), Design principles (Lecture 7), Selection/filtering (Lecture 5)

3. Risk-Return Scatter Plot (Enhanced)

Multi-interaction scatter with zoom, Voronoi, and convex hulls

Advanced Interactions:

- **d3.zoom:** Scroll to zoom (0.8x–6x), drag to pan, double-click resets
- **d3.Delaunay Voronoi:** Tessellation overlay provides large click/hover targets — no pixel-hunting small dots
- **Convex Hulls:** Semi-transparent boundaries around each category (15% padding) with centered labels
- **Smart Tooltips:** Rich cards with category badge, return/volatility/Sharpe/quadrant label, auto-positioned to stay on-screen
- **Clip Path:** Prevents overflow during zoomed state

Design Rationale: Quadrant lines at median risk/return divide chart into four zones (Low Risk High Return being optimal). Point size encodes Sharpe Ratio, allowing users to identify efficiently-priced assets at a glance.

Course Concepts: Zoom/pan/Voronoi (Lecture 5), Size/position encoding (Lecture 6), Scatterplot design (Lecture 11)

4. Historical Events with Impact Zones

Contextual market event annotations with temporal impact visualization

5 Annotated Events: ICO Boom (2017), COVID Crash (2020), Bull Peak (2021), Terra Collapse (2022), BTC Halving (2024)

Visual Features:

- Colored impact zones showing event duration (green=bullish, red=bearish, blue=neutral)
- Pulse animation (CSS animate-ping) on dots with active zones
- Crosshair line on hover for precise temporal reference
- Click-to-lock tooltips with rich market impact descriptions
- Impact period metadata (duration in months) in tooltip footer
- Toggle button to show/hide zones independently

Technical Challenge: Recharts intercepts pointer events on its wrapper div. We solved this by rendering interactive elements as React DOM divs with explicit pointerEvents:auto and z-index layering, using SVG only for non-interactive dashed lines.

Course Concepts: Annotation (Lecture 12), Layered visualization (Lecture 7), Event handling (Lecture 5)

5. Correlation Matrix & 6. DCA Performance Chart

10×10 Interactive Heatmap

Blue(-1) → Dark(0) → Red(+1) diverging color scale. Category filters, row/column highlighting on hover, interpretation text in tooltip. Pearson correlation from monthly returns.

DCA Performance Chart

Recharts LineChart comparing total invested (dashed) vs portfolio value (solid) over time for any of 21 assets. Shows power of dollar-cost averaging across 60 months of real price data.

Technical Implementation

Technology Stack

Framework: Next.js 16 (App Router)

Language: TypeScript + React 19

Visualization: D3.js v7 (custom)

Charts: Recharts (line charts)

Styling: Tailwind CSS v4

Onboarding: react-joyride v3

Export: jsPDF + html2canvas

Hosting: Vercel (Edge Network)

Data Pipeline

All data is sourced from public financial APIs and verified against multiple sources:

- **Price Data:** Monthly closing prices for 21 assets (2020–2024) for DCA calculator, annual normalized returns (2014–2024) for main comparator
- **Risk Metrics:** Computed client-side from monthly returns using our `advancedMetrics.ts` library (15 indicators including VaR, CVaR, Sortino, Treynor, Information Ratio)
- **Correlation Matrix:** Pearson coefficients computed from 60 months of returns, stored as JSON
- **Efficient Frontier:** Monte Carlo simulation runs in-browser (8000 portfolios, ~200ms computation)
- **Risk-Free Rate:** Monthly 3-month T-bill rates from FRED (2014–2024)

Performance Considerations

- **Dynamic imports** with `ssr: false` for D3 components (avoids server-side rendering of browser-dependent code)
- **ResizeObserver** for responsive chart sizing without expensive window resize listeners
- **useCallback memoization** prevents unnecessary re-renders of expensive chart drawing functions

- **Static pre-rendering** for all pages (Next.js generates HTML at build time)
- **Separated setup/update** in animated timeline (only transitions data-dependent elements, not static axes/grid)

Architecture Diagram

Layer	Components	Purpose
Pages	/, /risk-analysis, /dca-calculator, /portfolio-optimizer, /asset-evolution	Route-level layouts and data orchestration
Components	EfficientFrontier, AnimatedTimeline, RiskReturnScatter, CorrelationMatrix, HistoricalEvents	D3.js visualizations (client-only)
Libraries	portfolioOptimizer.ts, advancedMetrics.ts, pdfExport.ts	Computation and export logic
Data	/public/data/*.json	Pre-computed datasets

Evaluation and User Testing

Evaluation Methodology

We evaluated our visualization through two complementary approaches:

1. **Heuristic Evaluation:** Applied Shneiderman's Information Seeking Mantra (Overview first, zoom and filter, then details-on-demand) to each visualization
2. **Informal User Testing:** 5 EPFL students (non-finance background) were asked to complete tasks using the tool

Task-Based Evaluation Results

Task	Success Rate	Avg. Time	Notes
Compare Bitcoin vs S&P 500 (5y)	5/5	15s	Intuitive; asset selection clear
Find lowest-risk asset	5/5	25s	Used scatter plot quadrants
Understand what Sharpe Ratio means	5/5	8s	Tooltip immediately helpful
Build optimal portfolio with 5 assets	4/5	45s	One user missed the optimizer page initially
Identify COVID-19 market impact	5/5	12s	Impact zones made this obvious

Key Findings

- **Tooltips are essential:** All users relied on educational tooltips; without them, Sharpe Ratio and VaR were confusing
- **Guided tour effective:** Users who completed the tour navigated 40% faster on subsequent tasks
- **Category filters reduce cognitive load:** Users preferred filtering to 1-2 categories before comparing

- **Animation is engaging but not essential:** The timeline was the “wow factor” but users spent more time on scatter plot and optimizer
- **Impact zones add context:** “Oh, so that red zone explains why everything dropped!” — common reaction

Improvements Made from Testing

- Added “Double-click to reset zoom” hint after observing users stuck in zoomed state
- Increased font size in tooltips from 11px to 13px based on readability feedback
- Added navigation links between all pages (users were losing context between sections)
- Made guided tour contrast higher (white text on dark tooltips was too subtle initially)

Use of AI Tools

> **Transparency Statement:** We used AI tools (Claude Code / Claude Opus) as a coding assistant throughout development. Below we document exactly how AI was used, what it contributed, and what required human judgment.

How AI Was Used

Task	AI Contribution	Human Contribution
D3.js visualizations	Generated boilerplate, axis setup, scale computation	Design decisions, interaction design, visual encoding choices, debugging animations
Data preparation	Formatted JSON files, computed correlations and risk metrics	Sourced and verified data, chose time periods, defined metrics
React components	Component structure, state management, effect hooks	UX design, user flow, responsive breakpoints, accessibility
CSS/Tailwind styling	Utility class suggestions	Color palette, visual hierarchy, spacing decisions
Bug fixing	Diagnosed TypeScript errors, ResizeObserver issues	Identified root causes, decided on solutions, tested in browser

What AI Could NOT Do

- **Design decisions:** Choosing which visualizations best answer our research questions
- **User experience:** Understanding beginner mental models, deciding progressive disclosure levels
- **Data verification:** Confirming accuracy of financial data against known sources
- **Evaluation:** Conducting user tests, interpreting qualitative feedback

- **Creative direction:** The Hans Rosling-style timeline concept, impact zones idea, educational approach

Challenges and Solutions

Challenge	Solution
Recharts blocks pointer events for D3 overlays	Used React DOM elements with explicit pointerEvents:auto instead of D3 SVG elements
Animated timeline was jerky (full re-render per frame)	Separated static setup from dynamic update; used D3 enter/update/exit pattern
Efficient frontier without QP solver	Monte Carlo simulation with directed sampling covers feasible set adequately
21 assets in DCA with no API access	Manually compiled and cross-referenced monthly price data from public sources
Zoom state persists after filter change	Reset zoom transform on filter toggle; store zoom state in React state for UI feedback

Conclusion and Future Work

Project Achievements

The Investment Comparator successfully delivers on its core promise: making sophisticated financial comparison accessible to beginners. Key achievements include:

- **6 interactive D3.js visualizations** demonstrating diverse techniques (animation, zoom/pan, Voronoi, convex hulls, heatmaps, scatter plots)
- **21 real assets** with verified historical data spanning 11 years
- **Educational layer** with 17 tooltips, guided tour, and help system
- **Professional-grade features:** PDF export, portfolio optimization, 15+ risk metrics
- **Responsive design** working across desktop, tablet, and mobile

Lessons Learned

1. **Animation requires architecture:** Smooth D3 transitions demand separating static structure from dynamic data. Simply re-calling draw() per frame produces jank.
2. **Voronoi transforms interaction quality:** Adding invisible Voronoi cells made scatter plot interaction dramatically more usable—a small code change with outsized UX impact.
3. **Tooltips are the primary learning tool:** Users learn financial concepts through contextual tooltips far more effectively than separate documentation.
4. **Progressive disclosure works:** Showing 6 metrics by default and 15 on expand prevents overwhelming beginners while serving advanced users.

Future Work

- Real-time data integration via financial APIs (currently uses static historical data)
- User accounts with saved portfolios and watchlists
- Multi-asset DCA comparison (side-by-side DCA strategies)

- Machine learning-based return predictions (with appropriate disclaimers)
- Accessibility audit and WCAG 2.1 AA compliance verification

Team Contributions

Member	Primary Responsibilities
Nicholas Brandstätter	Frontend architecture, D3.js visualizations, data pipeline, deployment
Marko Djuric	Data sourcing & verification, financial metrics implementation, user testing
Toufan Kashaev	UX design, educational content, process book, evaluation methodology

---